# CoderUJB

## Un benchmark Java pour évaluer les LLM

Zeng et al. – ISSTA 2024

Présenté par

Wei WEI Hao QIAN Jianye SHI

Xiang BIAN Zijie Fang Yupan Huang

Master CHPS – UVSQ

Évaluation de performances

Article de Zhengran Zeng, Yidong Wang, Rui Xie, Wei Ye et Shikun Zhang

# Sommaire

1. Contexte de la recherche

2. Contributions et méthode

3. Résultats : contexte et modèles

4. Résultats : effets de l'entraînement

5. Limites de l'étude

6. Critique et conclusion

# 1. Contexte de la recherche

Wei WEI

# Comment évaluer une IA qui code ?

**Les LLM peuvent aider à programmer**

Proposer du code · Corriger une erreur

### À LA LECTURE

Un code peut **sembler correct**

**Il faut donc le tester.**

### À L'EXÉCUTION

Pourtant, il peut **échouer**

**Pour comparer les modèles : un benchmark**

Des tâches et des règles communes

# Un exercice isolé a ses limites

### HUMANEVAL

**164 exercices Python**

Compléter une fonction

Des tests vérifient le résultat

Mais les exercices restent
**assez isolés.**

### DANS UN VRAI PROJET

D'autres fichiers

Des bibliothèques

**Du code qui existe déjà**

**Exercice de cours $\neq$ logiciel complet**

Réussir le premier ne garantit pas qu'on réussira le second.

# Développer, c'est faire plusieurs tâches

**Écrire du code** **Créer des tests** **Trouver des erreurs** **Les corriger**

Une seule tâche donne une **vision limitée.**

## PLUSIEURS TÂCHES, MAIS D'AUTRES LIMITES

Selon les auteurs

| | |
|---|---|
| **CodeXGLUE** | Ressemblance entre les textes |
| **XCodeEval** | Problèmes de concours |

Ces approches ne réunissent pas toutes les conditions recherchées.

# Le besoin : une évaluation plus réaliste

**Réunir trois éléments**

**Exécuter les solutions**

Vérifier leur fonctionnement

**De vrais projets**

Code et environnement

**Plusieurs types de tâches**

Des capacités variées

**CoderUJB** – un benchmark Java

Comment mieux évaluer les capacités des modèles dans des situations proches du travail réel ?

# 2. Contributions et méthode

Hao QIAN

# Un benchmark commun pour cinq tâches

| 2 239 | 17 | 5 |
|---|---|---|
| questions | projets Java | tâches |

Questions issues de Defects4J, avec les projets et leur environnement d'exécution.

# Les cinq tâches de CoderUJB

| Tâche | Action demandée | Questions |
|---|---|---|
| FCG | Compléter une fonction | 238 |
| CTG | Créer un test à partir du code | 140 |
| ITG | Écrire un test reproduisant un bug | 451 |
| APR | Corriger une fonction | 470 |
| DD | Détecter un bug : oui / non | 940 |

Total : 2 239 questions. DD comprend 470 versions défectueuses et 470 versions corrigées.

# Construction des questions et évaluation

## Sélection

- Extraire le code et les tests, filtrer les questions, puis les vérifier

## Prompt

- Fournir la consigne et le contexte utiles à la tâche

## Évaluation

- Replacer les réponses dans le projet et les évaluer

Le projet complet sert à l'exécution. Le prompt n'en contient qu'une partie.

# Comment les réponses sont évaluées

## Génération

- 20 propositions par question ; pass-all@1 et pass-all@10

## Tests générés

- CTG : le test doit s'exécuter et couvrir le code testé
- ITG : échec sur la version défectueuse, succès après correction

## Détection

- Accuracy : proportion de réponses correctes

La syntaxe, la compilation et les tests sont vérifiés séparément.

# 3. Résultats : contexte et modèles

Jianye SHI

# Le contexte du projet aide la génération

```java
package org.apache.commons.math3.stat.regression;
import org.apache.commons.math3.linear.LUDecomposition;
public class GLSMultipleLinearRegression extends AbstractMultipleLinearRegression {
    private RealMatrix Omega;
    private RealMatrix OmegaInverse;

    protected RealVector calculateBeta();
    protected RealMatrix calculateBetaVariance();
}
```

**Import Context**

**Field Context**

**Signature Context**

Exemple de contexte : imports, champs de la classe et signatures des méthodes.

**CodeLlama-13B**

Génération de fonctions (FCG)

Le contexte du projet donne ici un meilleur score que les exemples.

| **Prompt** | **pass-all@1** |
|---|---|
| Un exemple | 12,84 % |
| Quatre exemples | 14,50 % |
| Contexte du projet | 21,91 % |

Source : Zeng et al., ISSTA 2024, fig. 2(a), extrait ; tableau 3.

# Un écart plus marqué en réparation

| **Modèle** | **FCG** | **APR** |
|---|---|---|
| CodeLlama-34B | 22,82 % | 5,01 % |
| GPT-3.5-Turbo | 23,37 % | 9,31 % |
| GPT-4 | 30,52 % | 18,76 % |

Scores : pass-all@1. Versions des modèles évaluées dans l'article de 2024.

# GPT-4 : des scores différents selon la tâche

| **GPT-4 : tâche** | **pass-all@1** |
|---|---|
| FCG : compléter une fonction | 30,52 % |
| CTG : écrire un test à partir du code | 24,18 % |
| ITG : écrire un test qui reproduit un bug | 6,66 % |
| APR : réparer une fonction | 18,76 % |

Scores à $k = 1$. Chaque tâche a ses propres critères de réussite.

# Détecter les bugs reste difficile

| 940 | 50 % | 52,16 % |
| --- | --- | --- |
| fonctions classées | toujours la même réponse | bonnes réponses de GPT-4 |

470 fonctions avec un bug et 470 versions corrigées.

# 4. Résultats : effets de l'entraînement

Xiang BIAN

# Deux façons de poursuivre l'entraînement

## RQ3 : poursuivre le préentraînement

- Continuer l'entraînement avec du code Python ou Java

## RQ4 : apprendre à suivre des instructions

- Entraîner le modèle sur des exemples de demandes et de réponses

Comparaison avec le modèle de départ, sur les mêmes tâches.

# Se spécialiser dans un langage : un compromis

**StarCoder-15B**

| **CoderUJB FCG / Java** | | **HumanEval / Python** | |
|---|---|---|---|
| Base | 15,32 % | Base | 30,35 % |
| + Python | 14,39 % | + Python | 33,57 % |
| + Java | 18,82 % | + Java | 27,07 % |

En génération de fonctions, chaque version progresse dans le langage ciblé.

Pour ITG et APR, les auteurs trouvent des résultats moins réguliers (RQ3).

# CodeLlama-13B après ajustement sur instructions

**CodeLlama-13B**

| Tâche et indicateur | Base | Instruct |
|---|---|---|
| FCG : pass-all@1 | 21,91 % | 13,28 % |
| CTG : pass-all@1 | 12,61 % | 6,14 % |
| APR : pass-all@1 | 4,50 % | 4,09 % |
| APR : pass-all@10 | 8,39 % | 15,72 % |

**Tableau 5 : syntaxe et compilation**
FCG, CodeLlama-13B

| Indicateur | Base | Instruct |
|---|---|---|
| Syntaxe | 94,14 % | 95,99 % |
| Compilation | 63,49 % | 40,76 % |

Le taux de syntaxe correcte reste proche ; le taux de compilation baisse.

# L'effet de l'entraînement dépend de la tâche

## RQ3 : préentraînement par langage

En génération de fonctions, les gains dépendent du langage. Les effets sont moins réguliers en ITG et APR.

## RQ4 : ajustement sur instructions

Les scores peuvent baisser en génération. En réparation, le gain dépend aussi du nombre de propositions.

## Interprétation des auteurs

Ces différences pourraient venir du lien entre la tâche évaluée et le préentraînement.

Cas particulier : CodeLlama-Instruct-34B produit des réponses répétées en FCG et CTG (mode collapse).

# 5. Limites de l'étude

Zijie Fang

# Les limites discutées par les auteurs

## Validité interne

- Erreurs possibles dans les scripts

## Validité externe

- Choix des modèles et généralisation

## Validité des mesures

- Indicateurs et qualité des tests

Ces trois limites sont discutées dans la section 6 de l'article.

# Ce que les tests ne vérifient pas

Les auteurs relisent leurs scripts et publient les résultats.

## Des bugs peuvent échapper aux tests

Un programme peut réussir les tests disponibles tout en contenant une erreur non testée.

## La couverture indique les lignes exécutées

Elle ne garantit pas que tous les comportements sont vérifiés.

# Des résultats sur un ensemble précis de projets

## Modèles

- Les versions de modèles retenues pour l'étude

## Données

- 17 projets Java issus de Defects4J

## Réparation (APR)

- Des bugs dans une seule fonction

La réparation de bugs répartis sur plusieurs fichiers n'est pas évaluée ici.

# Le protocole de génération étudié

## Protocole étudié

- Prompt → réponse → évaluation

## Autre approche possible

- Tester → corriger → recommencer

## Contexte fourni

- Le modèle ne reçoit pas forcément tout le projet

Les boucles de test et de correction restent à évaluer.

# 6. Critique et conclusion

Yupan Huang

# Ce que nous retenons de CoderUJB

## Des tâches réunies

- Générer du code, écrire des tests, détecter et réparer des bugs

## Des réponses exécutées

- Vérifier les réponses dans leur projet, selon les critères de chaque tâche

## Des résultats accessibles

- Code, scripts et résultats rendus publics

Nous retenons surtout la comparaison des résultats par tâche.

# Séparer l'effet du modèle et celui du prompt

## Constat du protocole

- Base : complétion ; Instruct : dialogue

## Notre réserve

- Le format peut contribuer à l'écart observé

## Amélioration proposée

- Comparer les formats, répéter les mesures et estimer l'incertitude

# GPT-4 en ITG : deux valeurs dans l'article

**GPT-4, tâche ITG**

| Indicateur | Valeur du tableau 4 |
|---|---|
| pass-all@1 | 6,66 % |
| pass-all@10 | 15,76 % |

RQ2 mentionne 15,76 parmi des scores annoncés à $k = 1$. Nos diapositives suivent le tableau 4.

# Conclusion

## Bilan

- CoderUJB compare plusieurs tâches dans des projets existants
- Les effets de l'entraînement varient selon la tâche
- Les scores se comparent à tâche et métrique identiques

## À tester ensuite

- D'autres projets et langages, avec des méthodes qui utilisent des outils
- Le risque de contamination et la sensibilité au format du prompt